

application layer

PROCESSES COMMUNICATING

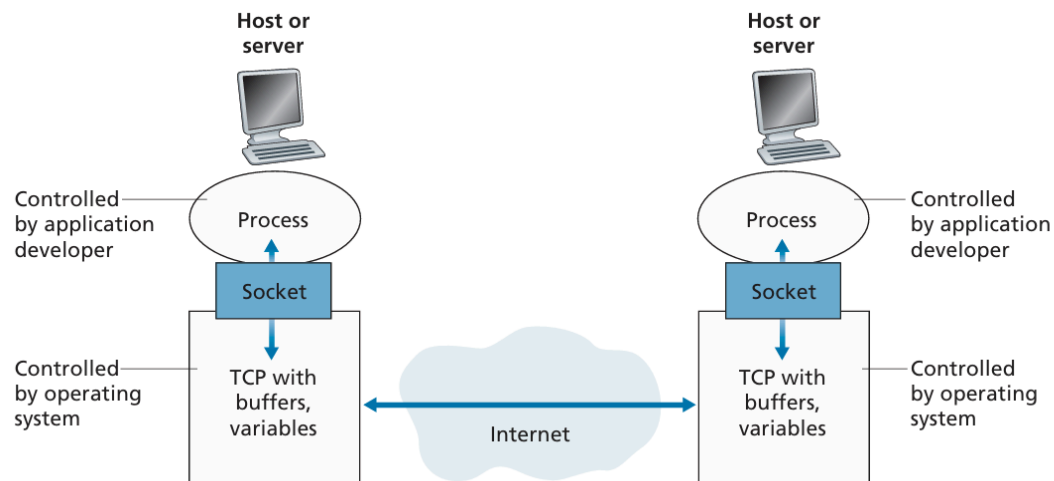


Figure 2.3 ♦ Application processes, sockets, and underlying transport protocol

Processes and Communication

- In networking, it is not the programs themselves but their **processes** that communicate.
- A **process** is a running instance of a program within an end system.
- If processes are on the **same host**, they communicate using **interprocess communication**, controlled by the operating system.
- Our main interest is in **processes on different hosts** communicating across a network.

Message Exchange

- Communication between remote processes occurs through the **exchange of messages**.
 - **Sending process**: creates and transmits a message into the network.
 - **Receiving process**: receives the message and may reply with another message.

- These processes operate in the **application layer** of the network protocol stack.
-

CLIENT AND SERVER PROCESSES

- A **network application** consists of communicating process pairs.
 - Example:
 - **Web** → Client: browser process | Server: web server process.
 - **P2P file sharing** → The peer downloading is the **client**, and the peer uploading is the **server**.
 - **Key Rule:**
 - The **client** is the process that **initiates contact**.
 - The **server** is the process that **waits to be contacted**.
 - **Note:**
 - In **P2P systems**, the same process can act as both client and server (uploading and downloading).
 - For any given communication session, roles are clearly defined: one is client, the other is server.
 - Often we use the terms **client side** and **server side** of an application.
-

INTERFACE BETWEEN PROCESS AND NETWORK (SOCKETS)

- A process communicates with the network through a **socket**.
- **Analogy:**

- Process = **house**
 - Socket = **door**
 - Messages exit one process through its socket and enter the other process through its socket.
 - **How it works:**
 - A sending process sends data through its socket into the network.
 - The transport layer delivers it across the Internet.
 - The receiving socket delivers it to the receiving process.
 - **Role of the Socket:**
 - The **socket** is the interface between the **application layer** and the **transport layer**.
 - Also called the **Application Programming Interface (API)** for network applications.
 - **Control Levels:**
 - **Application developer controls:** everything on the application side of the socket.
 - **Operating system controls:** transport protocol operations (e.g., TCP buffers, variables).
 - Developer's limited transport-layer control includes:
 1. Choosing the **transport protocol** (e.g., TCP, UDP).
 2. Possibly setting parameters like **buffer size** and **segment size**.
-

Conclusion

- Processes communicate by sending **messages** through **sockets**, which connect the application layer to the transport layer.

- The client-server relationship depends on who initiates contact.
- Developers mainly focus on the application side, while the operating system manages the underlying transport mechanisms.

TRANSPORT SERVICES PROVIDED BY THE INTERNET

Overview

The Internet provides two main transport protocols for applications: **UDP** and **TCP**. When creating a network application, developers must decide which one to use, based on the services required. Each protocol offers a distinct service model.

TCP SERVICES

1. Connection-Oriented Service

- Before exchanging application data, TCP requires the **client and server** to perform a **handshaking process**.
 - **Purpose:**
 - Establishes mutual awareness between the two processes.
 - Prepares them for sending and receiving large amounts of data.
 - Once established, a **TCP connection** exists between the sockets of the two processes.
 - **Full-duplex connection:**
 - Both processes can send and receive data **simultaneously**.
 - When communication ends, the connection must be **terminated (torn down)**.
-

2. Reliable Data Transfer Service

- TCP guarantees that all data sent is delivered:
 - **Completely** → No missing bytes.
 - **Accurately** → No errors.
 - **In order** → Bytes arrive in the same sequence as sent.
 - When one process sends a stream of bytes into its socket, the receiving process gets the same stream without loss or duplication.
-

3. Congestion Control

- TCP includes a **congestion-control mechanism**.
 - **Purpose:**
 - Not primarily for the communicating applications, but for the **overall health of the Internet**.
 - Prevents a single sender from overwhelming the network during congestion.
 - Ensures each TCP connection gets its **fair share of bandwidth**.
 - Mechanism: TCP throttles (slows down) the sending rate when congestion is detected.
-

Conclusion

TCP provides a **connection-oriented**, **reliable**, and **fair** transport service. It is best suited for applications that require guaranteed delivery and accuracy, such as **web browsing, email, file transfers, and financial transactions**.

(UDP services will be covered separately in this section.)

Application	Data Loss	Throughput	Time-Sensitive
File transfer/download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony/ Video conferencing	Loss-tolerant	Audio: few kbps–1 Mbps Video: 10 kbps–5 Mbps	Yes: 100s of msec
Streaming stored audio/video	Loss-tolerant	Same as above	Yes: few seconds
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes: 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

Figure 2.4 ♦ Requirements of selected network applications

UDP SERVICES

Overview

UDP (**U**ser **D**atagram **P**rotocol) is a simple, lightweight transport protocol that provides only the most basic communication services. Unlike TCP, it avoids overhead and complexity, making it fast but unreliable.

KEY CHARACTERISTICS

1. Connectionless

- UDP is **connectionless**, meaning:
 - No **handshaking** is performed before communication starts.
 - The sender can immediately transmit data to the receiver without setup.

2. Unreliable Data Transfer

- UDP does **not guarantee delivery** of messages.

- Possible issues:
 - Packets may be lost in the network.
 - Packets may arrive **out of order**.
 - No retransmission or error correction is provided.
- Applications using UDP must handle reliability themselves, if needed.

3. No Congestion Control

- UDP allows the sender to transmit data at **any rate** it chooses.
- However, actual throughput is limited by:
 - Network link capacities.
 - Network congestion.
- Unlike TCP, UDP does not slow down in response to congestion.

Conclusion

UDP offers **speed and simplicity** but sacrifices **reliability, ordering, and congestion control**. It is ideal for applications that:

- Can tolerate data loss (e.g., **multimedia streaming, VoIP, gaming**).
- Require **low latency** rather than guaranteed delivery.
- Prefer to manage reliability themselves rather than rely on the transport layer.

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP 1.1 [RFC 7230]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP (e.g., YouTube), DASH	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary (e.g., Skype)	UDP or TCP

APPLICATION-LAYER PROTOCOLS

Overview

Network processes communicate by exchanging messages through sockets. However, to ensure these messages are meaningful and understandable, their **structure, meaning, and exchange rules** must be defined. This is the role of **application-layer protocols**.

WHAT AN APPLICATION-LAYER PROTOCOL DEFINES

1. Types of Messages Exchanged

- Specifies whether messages are **requests**, **responses**, or other forms.

2. Syntax of Messages

- Defines the **fields in each message** and how they are separated or structured.

3. Semantics of Fields

- Explains the **meaning of each field** and the information it conveys.

4. Rules of Communication

- Determines **when** a process should send a message.
- Specifies how a process should **respond** to messages it receives.

PUBLIC VS. PROPRIETARY PROTOCOLS

- **Public Protocols:**
 - Published in **RFCs (Request for Comments)** and available for everyone.
 - Example: **HTTP (HyperText Transfer Protocol, RFC 7230)**.
 - Benefit: If all developers follow the HTTP RFC, any **browser** can retrieve web pages from any **web server** that follows the same standard.
- **Proprietary Protocols:**
 - Not publicly available; intentionally kept private by organizations.
 - Example: **Skype** uses proprietary application-layer protocols.

APPLICATIONS VS. APPLICATION-LAYER PROTOCOLS

It is essential to distinguish between the **network application** itself and the **protocol** it uses. The protocol is just **one part** of the application.

Example 1: The Web

- Components include:
 - **Document formats** → HTML.
 - **Clients** → Browsers (e.g., Chrome, Internet Explorer).
 - **Servers** → Web servers (e.g., Apache, Microsoft servers).
 - **Application-layer protocol** → HTTP.
- Role of HTTP: Defines **format and sequence of messages** between browser and server.

- Note: HTTP is important, but it is only **one piece** of the entire Web system.

Example 2: Netflix

- Components include:
 - **Servers** → Store and transmit videos.
 - **Billing/management servers** → Handle subscriptions, accounts, and user data.
 - **Clients** → Netflix app on smartphones, tablets, PCs, smart TVs.
 - **Application-layer protocol** → **DASH (Dynamic Adaptive Streaming over HTTP)**.
 - Role of DASH: Defines the **format and sequence of messages** between Netflix servers and clients for video streaming.
 - Again, DASH is only **one component** of the broader Netflix application ecosystem.
-

Conclusion

- An **application-layer protocol** is crucial for enabling communication between distributed processes.
- It defines **message types, syntax, semantics, and rules** of communication.
- While critical, it is only a **part of a larger network application**, which usually includes multiple components such as servers, clients, formats, and services.

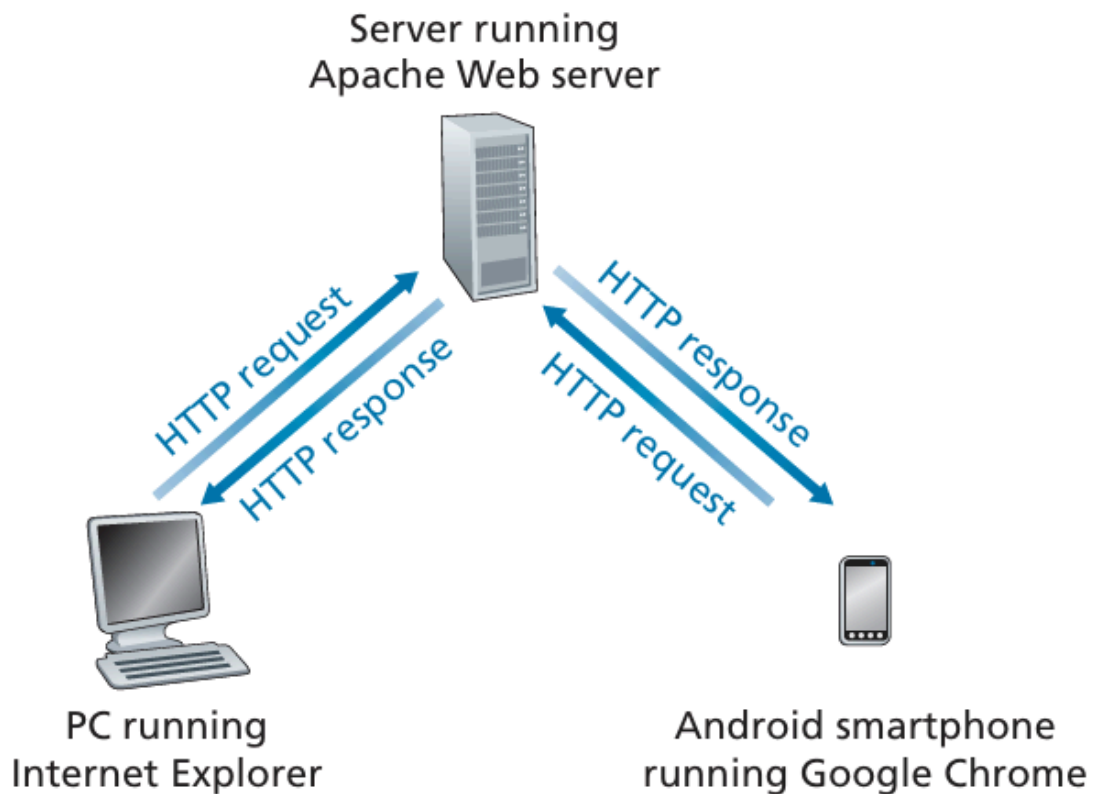
OVERVIEW OF HTTP

Introduction

The **HyperText Transfer Protocol (HTTP)** is the Web's **application-layer protocol**. It is defined in several RFCs, including **RFC 1945**, **RFC 7230**, and **RFC 7540**. HTTP operates with two programs:

- **Client program** (e.g., a browser like Chrome, Edge, or Firefox).
- **Server program** (e.g., Apache, Microsoft IIS).

These programs run on different systems and communicate by exchanging **HTTP messages**, whose structure and interaction rules are defined by the protocol.



WEB TERMINOLOGY

1. Web Page (Document)

- Consists of multiple **objects**.
- An **object** is any file addressable by a **URL**: examples include **HTML files**, **images (JPEG/GIF)**, **CSS files**, **JavaScript files**, **video clips**.
- A typical page includes:

- A **base HTML file**.
- Several **referenced objects** (images, scripts, etc.).

2. Example:

- A page with HTML text + 5 JPEG images = **6 objects** in total.
- The base HTML file contains **references (URLs)** to these additional objects.

3. **URL (Uniform Resource Locator)**

- Each URL has two components:
 - **Hostname** → the server address.
 - **Path name** → the specific location of the object.

Example:

`http://www.someSchool.edu/someDepartment/picture.gif`

- - Hostname: `www.someSchool.edu`
 - Pathname: `/someDepartment/picture.gif`

4. **Clients and Servers**

- **Browsers** implement the client side of HTTP.
- **Web servers** store objects and deliver them upon request. Popular servers: **Apache, Microsoft IIS**.

HTTP OPERATION

1. **Client-Server Interaction**

- When a user clicks a hyperlink, the browser sends an **HTTP request message** for each object in the page.
- The server replies with **HTTP response messages**, containing the requested objects.

2. Underlying Transport Protocol

- HTTP uses **TCP** for communication (not UDP).
- Process:
 - Browser (client) first establishes a **TCP connection** with the web server.
 - HTTP request messages are sent via the socket interface to TCP.
 - Server receives them and responds with HTTP response messages through its socket.
- TCP ensures **reliable data transfer**:
 - Every request reaches the server intact.
 - Every response reaches the client intact.
- Advantage of layering: HTTP does not worry about packet loss or reordering—TCP handles it.

3. Stateless Protocol

- HTTP does not maintain **state information** about clients.
- If the same client requests the same file multiple times, the server resends it without remembering previous requests.
- Because of this, HTTP is called a **stateless protocol**.

4. Architecture

- HTTP follows a **client-server architecture**.
- Web server: **always on**, has a fixed IP address, and can serve millions of clients simultaneously.

HTTP VERSIONS

1. HTTP/1.0 (Early 1990s, RFC 1945)

- The original version.

2. HTTP/1.1 (RFC 7230)

- The most widely used version (as of 2020, majority of traffic).

3. HTTP/2 (RFC 7540)

- Supported increasingly by modern browsers and web servers.
 - Introduces more efficient features for performance improvement.
-

Conclusion

HTTP is the **core protocol of the Web**, built on TCP for reliable communication. It defines how **clients (browsers)** and **servers** exchange structured request/response messages. Its **stateless nature** simplifies design but also requires additional mechanisms (like cookies and sessions) for stateful interactions. Over time, HTTP has evolved from **HTTP/1.0** to **HTTP/1.1** and now towards **HTTP/2**, enabling faster and more efficient web communication.

NON-PERSISTENT AND PERSISTENT CONNECTIONS

Overview

In many Internet applications, especially the Web, the client and server communicate repeatedly—sometimes back-to-back, sometimes periodically. When using TCP, the developer must decide:

- Should each **request/response pair** use a **separate TCP connection**?
(Non-persistent connections)

- Or should all requests/responses use a **single TCP connection?** (**Persistent connections**)

HTTP supports both approaches, though HTTP/1.0 used non-persistent by default, while HTTP/1.1 introduced persistent as the standard.

HTTP WITH NON-PERSISTENT CONNECTIONS

Step-by-Step Example

Suppose a web page consists of:

- **1 base HTML file.**
- **10 JPEG images.**
All objects are stored on the same server.

Steps:

1. TCP Connection Initiation:

- The client initiates a TCP connection to the server (www.someSchool.edu) on port **80** (default HTTP port).
- A socket is created at both client and server.

2. Request Sent:

- The client sends an **HTTP request message** for </someDepartment/home.index>.

3. Server Responds:

- The server retrieves the object, encapsulates it in an **HTTP response message**, and sends it back.

4. Connection Closed:

- The server instructs TCP to close the connection once the object is sent (after ensuring safe delivery).

5. Client Receives Response:

- The client extracts the HTML file, sees references to 10 JPEGs.

6. Repeat Process:

- Steps 1–4 are repeated **for each JPEG image**.

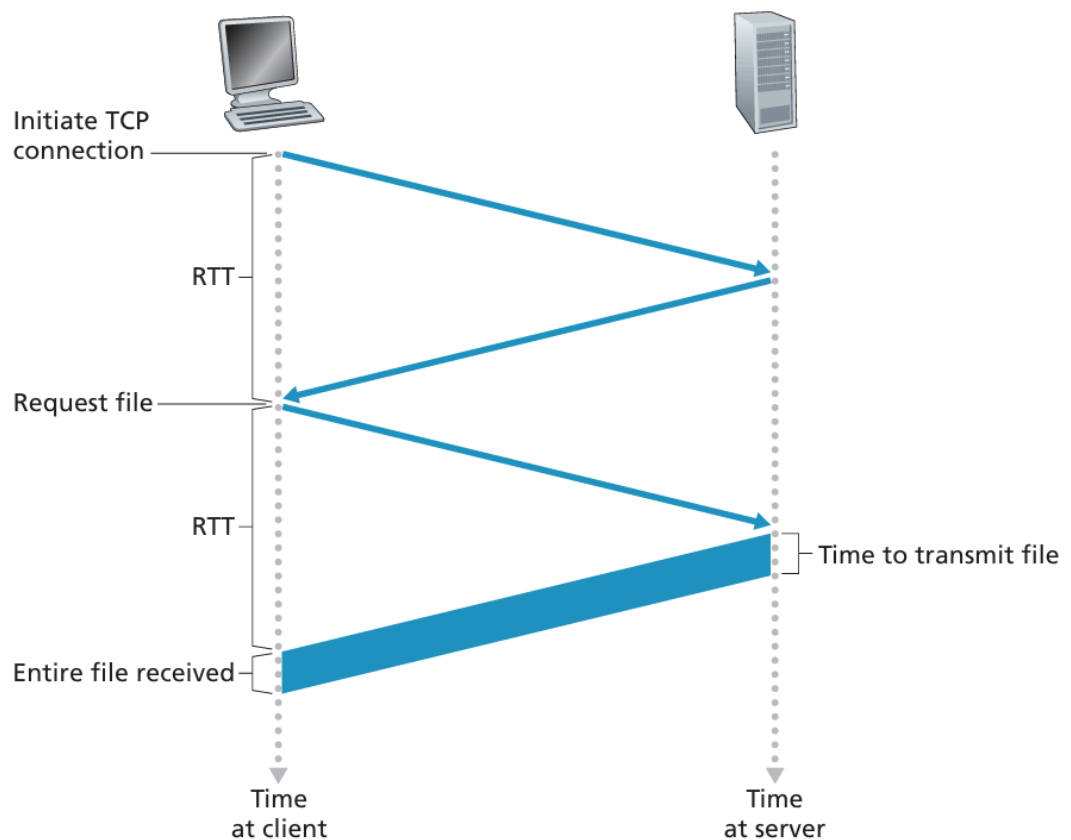


Figure 2.7 ♦ Back-of-the-envelope calculation for the time needed to request and receive an HTML file

Key Points:

- Each TCP connection carries **exactly one request and one response**.
- To fetch a page with 11 objects, **11 TCP connections** are required.

- Browsers may download objects:
 - **Serially** (one by one).
 - **In parallel** (multiple TCP connections open simultaneously), reducing total load time.
-

ROUND-TRIP TIME (RTT) AND RESPONSE DELAY

- **RTT** = Time for a small packet to go from client → server → client.
 - Includes propagation delay, queuing, and processing.

Sequence for a single HTML file (see Figure 2.7):

1. TCP Handshake:

- Three-way handshake requires **1 RTT** (first two steps).

2. Request & Response:

- The client sends the HTTP request (piggybacked on the third handshake step).
- The server responds with the file.
- This takes **1 more RTT**.

3. File Transmission:

- Server transmits the file.
- Transmission time depends on file size and link bandwidth.

Total Response Time $\approx$ 2 RTTs + File Transmission Time

SUMMARY

- **Non-persistent connections:** Each request/response uses a new TCP connection.
 - **Advantages:** Simple, clean separation between requests.
 - **Disadvantages:** High overhead (multiple connections → multiple RTTs).
- **Persistent connections** (covered next): Allow multiple objects to be transferred over a **single connection**, reducing RTT overhead and improving efficiency.

Would you like me to continue with **Persistent Connections** explanation as the next section (2.2.2 continuation), so both approaches are covered in one complete note?

HTTP WITH PERSISTENT CONNECTIONS

Limitations of Non-Persistent Connections

- For each object requested, a **new TCP connection** must be created.
 - This requires:
 - Allocating **TCP buffers** and maintaining **TCP variables** at both client and server.
 - High load on web servers handling **hundreds of simultaneous clients**.
 - Each object suffers a **2-RTT delay**:
 - **1 RTT** to establish the TCP connection.
 - **1 RTT** to request and receive the object.
 - Overall, non-persistent connections are inefficient, especially when multiple objects must be downloaded.
-

Persistent Connections in HTTP/1.1

1. Connection Reuse

- After sending a response, the server **keeps the TCP connection open**.
- The same connection is reused for subsequent requests and responses between the **same client and server**.

2. Efficiency

- A whole web page (base HTML + all embedded objects) can be fetched using **one persistent TCP connection**.
- Even multiple web pages from the same server can be retrieved over **one connection**.

3. Pipelining

- Clients can send **multiple requests back-to-back**, without waiting for earlier responses.
- Servers then reply with the requested objects **back-to-back**, reducing idle waiting time.

4. Connection Timeout

- If a connection remains idle beyond a configurable **timeout interval**, the server closes it.
- This prevents wasted resources from unused connections.

DEFAULT MODE

- Since HTTP/1.1, **persistent connections with pipelining** are the **default behavior**.
- This significantly reduces RTT overhead, improves response time, and lowers server load compared to non-persistent connections.

Conclusion

- **Non-persistent connections (HTTP/1.0):** Each object requires a separate TCP connection → high overhead and delay.
- **Persistent connections (HTTP/1.1 default):** Multiple objects (and even multiple web pages) can be transferred over a single TCP connection with pipelining, making the web faster and more efficient.

HTTP MESSAGE FORMAT

Overview

HTTP messages are defined in the specifications [RFC 1945; RFC 7230; RFC 7540]. There are **two types of HTTP messages**:

1. **Request messages** (sent by client to server).
2. **Response messages** (sent by server to client).

We start with **HTTP request messages**.

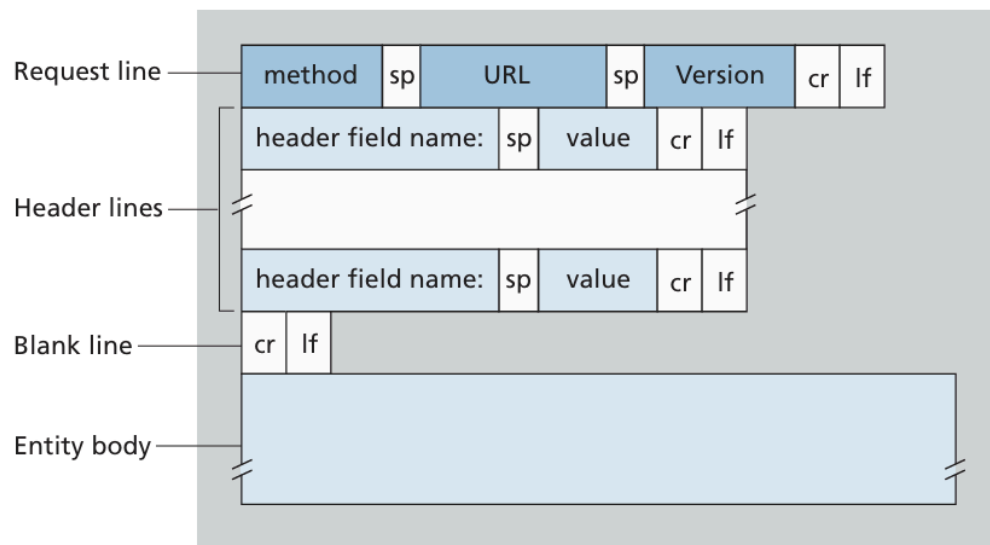


Figure 2.8 ♦ General format of an HTTP request message

EXAMPLE OF AN HTTP REQUEST MESSAGE

GET /somedir/page.html HTTP/1.1

Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr

Key Observations

1. **Human-readable** → Written in ASCII text.
 2. **Structure** → Composed of multiple lines, each ending with a **carriage return (CR)** and **line feed (LF)**.
 3. **Parts:**
 - **Request line** (first line): Contains method, URL, and HTTP version.
 - **Header lines**: Provide additional information about the request.
 - **Entity body**: Optional, used with certain methods (e.g., POST).
-

REQUEST LINE

- **Format:** `Method URL Version`
- **Fields:**
 1. **Method** → specifies action (GET, POST, HEAD, PUT, DELETE).
 2. **URL** → identifies requested object.
 3. **Version** → indicates HTTP version (e.g., HTTP/1.1).
- **Common Methods:**
 1. **GET** → retrieves an object (most common).
 2. **POST** → sends data to the server (often used with forms; data included in entity body).

3. **HEAD** → like GET, but only returns headers (no object). Useful for debugging.
 4. **PUT** → uploads an object to a specified path on the server.
 5. **DELETE** → removes an object from the server.
-

HEADER LINES (EXAMPLES)

- **Host:** www.someschool.edu
 - Specifies the host where the object resides.
 - Required especially for **web proxy caches**.
 - **Connection: close**
 - Tells the server to close the TCP connection after sending the response (non-persistent connection).
 - **User-agent: Mozilla/5.0**
 - Identifies the client software (browser type).
 - Allows the server to deliver content optimized for different browsers.
 - **Accept-language: fr**
 - Indicates preferred content language (here, French).
 - Part of **content negotiation** headers.
-

ENTITY BODY

- Present in some HTTP requests (mainly with **POST**).
- Contains **data submitted by the client** (e.g., form input values).

- Example: Submitting search words to a search engine.
- Alternative with **GET**:
 - Form data can also be appended to the **URL** itself.

Example:

`www.somesite.com/animalsearch?monkeys&bananas`

-
- Commonly seen in day-to-day browsing.

GENERAL FORMAT OF AN HTTP REQUEST MESSAGE (Figure 2.8)

1. **Request Line** → `Method URL Version`
2. **Header Lines** → Key-value pairs (e.g., Host, User-Agent).
3. **Blank Line** → Separates headers from body.
4. **Entity Body** → Optional data (mainly for POST).

HTTP RESPONSE MESSAGE

Overview

HTTP response messages are sent by the server to the client after processing a request. Each response includes a status line, a set of header lines, and an entity body containing the requested object (if available).

EXAMPLE RESPONSE MESSAGE

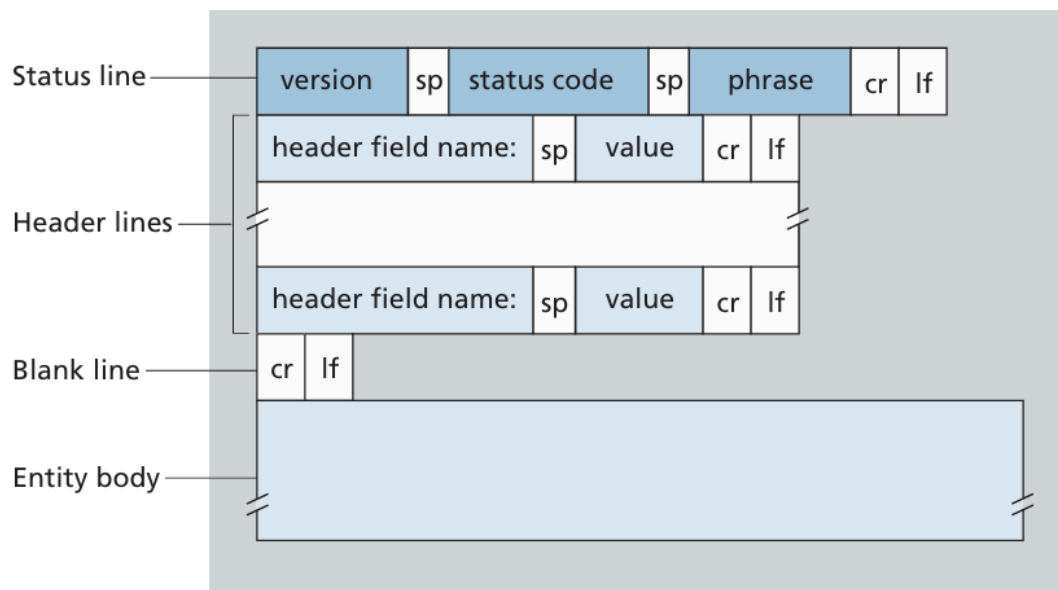


Figure 2.9 ♦ General format of an HTTP response message

HTTP/1.1 200 OK

Connection: close

Date: Tue, 18 Aug 2015 15:44:04 GMT

Server: Apache/2.2.3 (CentOS)

Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT

Content-Length: 6821

Content-Type: text/html

(data data data data data ...)

Structure

1. Status line → **HTTP version | status code | phrase.**
 - Example: **HTTP/1.1 200 OK** (indicating success).
2. Header lines → Provide metadata about the response.

- **Connection: close** → Server will close TCP connection after sending.
- **Date: ...** → Time when response was created and sent (not when file was originally created).
- **Server: Apache/2.2.3 (CentOS)** → Identifies the web server software.
- **Last-Modified: ...** → Timestamp of last modification of the requested object.
 - Important for caching (both client-side and proxy servers).
- **Content-Length: 6821** → Size of the object in bytes.
- **Content-Type: text/html** → Specifies type of object (HTML text in this case).

3. Entity Body

- Contains the actual requested object (HTML file, image, video, etc.).
- This is the “payload” of the response.

GENERAL FORMAT (Figure 2.9)

1. **Status Line** → **Version Status-Code Phrase**
2. **Header Lines** → Key-value pairs describing response.
3. **Blank Line** → Separates headers from body.
4. **Entity Body** → The actual content (may be empty for some responses, like HEAD).

COMMON STATUS CODES

- **200 OK** → Request succeeded; object is returned.

- **301 Moved Permanently** → Object moved; new URL in **Location:** header. Browser automatically redirects.
 - **400 Bad Request** → Generic error; server couldn't understand the request.
 - **404 Not Found** → Requested object does not exist on this server.
 - **505 HTTP Version Not Supported** → Server does not support the HTTP version used in the request.
-

PRACTICAL USE

Developers can use tools like Telnet to manually send HTTP requests and view raw response messages.

Example:

```
telnet gaia.cs.umass.edu 80
```

```
GET /kurose_ross/interactive/index.php HTTP/1.1
```

```
Host: gaia.cs.umass.edu
```

- - Returns an HTTP response with headers and the requested HTML object.
 - If **HEAD** is used instead of **GET**, the server returns headers only (without the object).
-

Conclusion

An HTTP response message provides both metadata (via the status line and headers) and the actual object (entity body). The status code and phrase indicate the outcome of the request, while headers supply crucial details like object size, type, and modification time. These features support reliable communication, caching, and error handling, making HTTP flexible and robust.

